

Agentic Pilot: An Evidence-Driven Local Autonomous AI Agent Framework for Privacy-Preserving Intelligent Task Automation

[TODO: Author name(s)]

[TODO: Affiliation, city, country]

[TODO: email]

Abstract—LLM-based assistants that operate software on a user’s behalf usually send screenshots and page content to hosted models, retain little from one session to the next, and can declare a task finished when it is not. We present Agentic Pilot, an open-source agent that runs its language and vision models locally through Ollama and drives a Chromium browser through Playwright. The agent is a LangGraph state machine in which observation, planning, execution, verification and recovery are separate nodes, and a task may be reported complete only when a completion predicate evaluated on the live page holds; each evaluation is stored with before/after screenshots and a DOM snapshot. Grounding uses a compact DOM manifest first and a local vision model only when the planner cannot act on the DOM. Calls are routed to small models by role, CAPTCHAs halt the task for the user, and high-risk intents require human approval. We formalize the gate and define a false completion rate. On 35 constructed browser end states, the gate withheld completion in 15 of the 22 states that did not satisfy their goal, all of which a dispatch-only rule would accept, and rejected 1 of 13 satisfied states, at a median cost of 33.8ms per decision; the seven misses identify specific missing checks. A code-level audit shows which mechanisms are active on the live path. We also design an extension to files, processes and native applications with accessibility-tree grounding, typed postconditions and deterministic permission tiers, and give an end-to-end evaluation protocol for local hardware.

Index Terms—autonomous agents, large language models, local inference, privacy-preserving computing, task automation, execution verification, human-in-the-loop

I. INTRODUCTION

Assistants built on large language models (LLMs) increasingly act on a user’s behalf: they open pages, fill forms and read results. Several of the web and GUI agents reviewed in Section II send each observation, often a full screenshot, to a hosted multimodal model [1]–[4]. For a personal assistant this is a poor fit. The screen of a logged-in user carries names, messages and account data, and leakage of personal information from language models has been measured even after scrubbing [5]. Few of these systems keep memory across sessions, and most operate either in a browser or in one other environment rather than across the applications a person uses (Table I).

A second problem is independent of where the model runs. An agent can believe it has finished when it has not. We call this *premature completion*: the agent emits a terminal “done” while the environment does not satisfy the user’s

goal. A typical case is a search request in which the agent opens the search engine, sees the page load without error, and stops before any query is issued. Every individual browser call succeeded; the task did not. Benchmarks that check final environment state show how wide this gap is [6], [7], and self-assessment by the model is not a reliable remedy, since intrinsic self-correction without external feedback does not improve, and can degrade, reasoning accuracy [8].

This paper describes Agentic Pilot, an open-source agent that runs its language and vision models locally through Ollama and drives a Chromium browser through Playwright. Its central design rule is that a task may be reported complete only when a check on the observed environment state passes, and that the inputs to this check are written to disk as evidence. The agent is structured as an explicit state machine in LangGraph, so that observation, planning, execution, verification and recovery are separate nodes whose transitions can be inspected and logged.

We make the following contributions:

- 1) An evidence-gated execution model that separates action dispatch from task completion, formalized as a completion predicate over observations, together with a *false completion rate* (FCR) metric for measuring premature completion (Section IV-C).
- 2) The architecture and implementation of a local agent that combines this gate with role-based routing across local models, DOM-first grounding with a vision-model fallback, CAPTCHA detection that halts rather than bypasses, a task-level human approval gate, and persistent per-step evidence (Sections III–V).
- 3) A controlled, reproducible study of the completion gate on 35 constructed browser end states, which quantifies both the premature completions it prevents and the ones it misses (Section VII).
- 4) A design for extending the framework from the browser to the local system (files, processes and native applications) with accessibility-tree-first grounding, typed desktop verification predicates and deterministic permission tiers; we state precisely which parts of it exist in code today (Section IV-G).
- 5) A code-level audit of which mechanisms are active on the live execution path, and an end-to-end evaluation protocol with ablations for the author’s hardware (Sec-

tions V and VI).

Section II reviews related work. Section III presents the architecture and Section IV the methodology, including the formal model and the local-system extension. Section V covers the implementation and its status, Section VI the experimental setup and Section VII the results. Sections VIII and IX discuss security and limitations, and Section X concludes.

II. RELATED WORK

A. Reasoning-and-acting agents

ReAct interleaves free-text reasoning with tool actions and observations [9], building on chain-of-thought prompting [10]. Toolformer shows that a model can learn when to call an external API [11]. Reflexion stores verbal self-critiques in an episodic buffer and retries with them [12]; RCI applies recursive self-criticism to computer tasks [13]; Voyager accumulates a skill library and uses a model-based self-check before committing a skill [14]. In all of these, the decision that a task is finished is made by the model, sometimes with a critic that is also a model. Huang et al. find that such intrinsic self-correction does not reliably improve reasoning without external feedback [8]. Agentic Pilot moves the completion decision out of the model and into a check on the observed page.

B. Web agents and benchmarks

Early web agents were trained with reinforcement or imitation learning on MiniWoB-style pages [15]. Mind2Web pairs a small ranking model that filters DOM candidates with an LLM that chooses the action [16], and WebArena evaluates agents on self-hosted sites with functional correctness checks [6]. WebVoyager drives live sites from screenshots and uses a multimodal model to judge saved trajectories after the episode [1]. The correctness checks in these works belong to the benchmark and run after the episode ends. Our gate runs inside the agent at the moment it tries to stop.

C. Desktop, OS and mobile GUI agents

OSWorld provides execution-based checkers for tasks in real operating systems and reports a large gap between agents and people [7]. UFO controls Windows applications through UI Automation and asks the user to confirm sensitive actions [3]. AppAgent learns reusable documentation of smartphone apps [4], and GPTDroid and QTypist apply LLMs to mobile GUI testing and text-input generation [17], [18]. These systems either rely on hosted models or target testing rather than verified completion of user tasks. Our desktop extension (Section IV-G) borrows UFO’s accessibility-first control and OSWorld’s state-based checking, but places the checks inside the agent loop.

D. Visual grounding for GUI interaction

SeeAct finds that grounding a chosen action onto the right element is the main bottleneck for multimodal web agents, and that combining HTML with the image outperforms Set-of-Mark overlays [2], [19]. CogAgent and SeeClick train

vision-language models that locate elements from screenshots alone [20], [21], and OmniParser converts screenshots into structured element lists [22]. These results motivate our ordering: structured DOM grounding first, pixel-level grounding only when the DOM route fails.

E. Agent memory and retrieval

Retrieval-augmented generation couples a model with a non-parametric store [23], usually indexed with sentence embeddings [24]. Generative Agents score memories by recency, importance and relevance [25], and MemGPT pages context between tiers in the manner of an operating system [26]. None of these ties a stored memory to verified evidence of the outcome it describes, which is what an execution agent needs if it is to learn from its own runs.

F. Security of LLM agents

Indirect prompt injection places instructions in content the agent reads, such as a web page [27]. Liu et al. formalize such attacks and benchmark defenses [28]; InjecAgent and AgentDojo measure how often tool-using agents follow injected instructions [29], [30]. A browser agent reads untrusted text at every step, so it needs both input filtering and a policy that keeps high-impact actions under human control.

G. Local and on-device inference

Open-weight families such as Qwen2.5 and Qwen2-VL provide text and vision models at sizes that run on consumer hardware [31], [32], and weight quantization further reduces their memory footprint [33]. Surveys of LLM agents [34], [35] note that dependable execution and deployment constraints remain open problems. Whether small local models can complete verified GUI tasks is largely unmeasured; our evaluation protocol is designed to measure it.

Table I summarizes the comparison. Cells record only what each paper reports.

III. SYSTEM ARCHITECTURE

Fig. 1 shows the components and the trust boundary. A desktop client submits a natural-language task to a FastAPI service bound to the loopback interface. The task runner queues it, executes the agent graph, checkpoints the state after every node so that a paused or interrupted task can resume, and cancels tasks that exceed a wall-clock limit (5 min by default).

The agent is a compiled LangGraph state machine with eleven nodes (Fig. 2). A typed state record carries the parsed intent, the plan and step index, the page manifest, the planned action, the action history, retry and model-call counters, routing decisions and the status; nodes return partial updates and the routers read only the status and a few flags. The single terminal node, `complete`, persists the outcome, writes memory records and decides whether the browser stays open for the user. Model calls go through a router (Section IV-E) and a gateway that requests JSON output from a local Ollama server, validates it against Pydantic schemas and re-prompts on malformed output. A pool of Playwright contexts over one

TABLE I

CAPABILITIES REPORTED BY REPRESENTATIVE SYSTEMS. LOC: RUNS ON OPEN OR LOCALLY HOSTABLE MODELS; WEB/DESK: TARGET ENVIRONMENT; GATE: COMPLETION GATED AT RUN TIME BY AN IN-AGENT ENVIRONMENT CHECK; EVID: PERSISTS STATE ARTIFACTS; MEM: CROSS-STEP OR CROSS-EPISEODE MEMORY; INJ: PROMPT-INJECTION DEFENSE; APPR: HUMAN APPROVAL FOR RISKY ACTIONS. **Y** REPORTED, **P** PARTIAL OR WEAKER FORM, **N** NOT SUPPORTED, – NOT REPORTED. THE LAST ROW REFLECTS THE AUDITED CODE (SECTION V).

System	Loc	Web	Desk	Gate	Evid	Mem	Inj	Appr
ReAct [9]	N	P	N	N	N	N	N	N
Reflexion [12]	–	P	N	P	N	Y	N	N
WebArena [6]	–	Y	N	P	–	N	N	N
Mind2Web [16]	Y	Y	N	N	P	N	N	N
WebVoyager [1]	N	Y	N	P	Y	N	N	N
SeeAct [2]	P	Y	N	N	–	N	N	P
OSWorld [7]	Y	Y	Y	P	–	N	N	N
UFO [3]	N	P	Y	–	–	P	N	Y
AppAgent [4]	N	N	N	–	–	Y	N	N
CogAgent [20]	Y	Y	–	N	N	N	N	N
MemGPT [26]	–	N	N	N	N	Y	N	N
AgentDojo [30]	Y	N	N	P	–	N	Y	–
Agentic Pilot	Y	Y	P ^a	Y ^b	Y	P ^c	P ^d	P ^e

^a Designed (Section IV-G); a prototype executor exists but is not wired into the agent.

^b Rule-based predicate Φ ; see its measured blind spots in Section VII. ^c Outcomes are written; retrieval is not active on the live path. ^d Pattern denylist on element text.

^e Task-level, triggered by the intent’s action type or model-assigned risk level.

AppAgent targets Android, which we do not count as desktop.

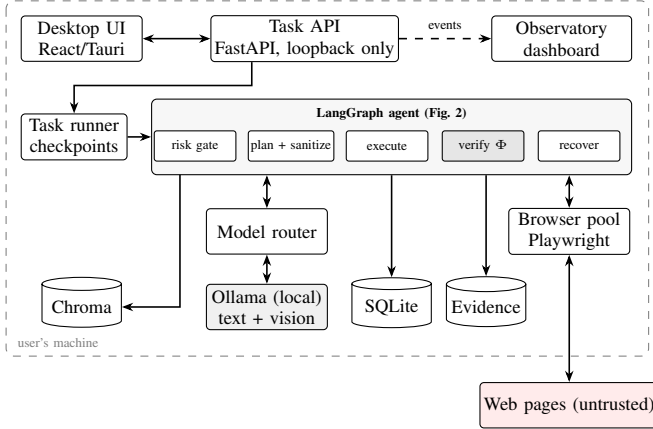


Fig. 1. Components of Agentic Pilot. Everything inside the dashed boundary runs on the user’s machine, including model inference. Web pages are the main untrusted input, and browsing is the main outbound traffic. The Observatory also reads the task database and evidence files.

headless Chromium gives each task its own page. Task events go to SQLite and to a broadcaster; the Observatory, a separate process, subscribes to that stream and reads the database and evidence files to display plans, routing decisions, verification outcomes and screenshots. It only issues read queries and has no command channel to the agent.

Two inputs cross into the agent: the user’s task text, which is trusted, and page content, which is not. Page content reaches the models as element text, page title and URL, and, on the vision path, screenshots. Apart from one-time downloads of model weights (the Ollama models and, on first use, the default embedding model of the vector store), outbound traffic in the

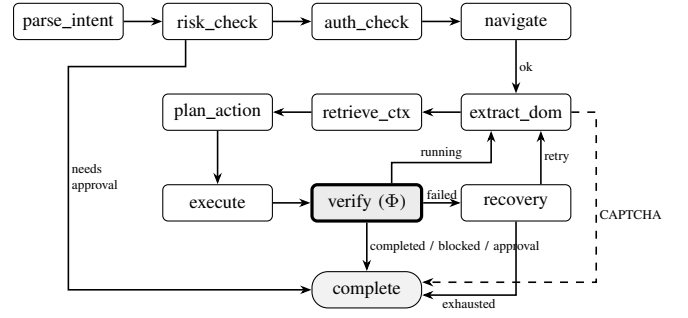


Fig. 2. Execution graph compiled in graph.py. Only the shaded verify node can end a task as COMPLETED, and only when Φ holds. A CAPTCHA seen during observation ends it as BLOCKED. Not drawn: failures in parse_intent and navigate also route to recovery.

default configuration consists of the browser’s requests to the sites a task visits; model traffic stays on the local host unless the user points the Ollama address elsewhere.

IV. METHODOLOGY

A. Task Decomposition and Planning

The parse_intent node asks a lightweight model for a structured intent: action, target site, target and content strings, risk level and confidence. If the request contains a connective (“and”, “then”, a comma) or more than eight words, a planning model decomposes it into steps $P = (g_1, \dots, g_n)$, each with a description, an action type and an expected outcome; a request to enter literal text becomes a single text-entry step, and anything else a one-step plan.

Execution then chooses one action per iteration. plan_action gives the model the goal, the plan with the current step marked, the page title, URL and state, and the element manifest, and asks for one of navigate, click, type_text, press_key, scroll, extract, complete or need_help. Deterministic rules then adjust the choice: exact-text requests are grounded to the best-scoring visible text field, repeated navigation to the current site is suppressed, and plan steps that name clicking a result, extracting or reporting are mapped to those actions. Some rules are site-specific; on Google with a search goal the agent types the query and presses Enter rather than loading a results URL directly (see Section VIII).

B. Grounding: DOM First, Vision on Demand

Each observation starts with a DOM pass over buttons, inputs, text areas, selects, links, elements with button or link roles, and editable regions. For each element the extractor records tag, role, accessible label, up to 120 characters of text, placeholder, type, visibility, enabled state, bounding box, and three selectors (an identifier attribute it writes into the live DOM, a CSS selector and an XPath, tried in that order). Only interactable elements are kept; they are ranked with submit-like controls first, deduplicated, limited to five links, stripped of cookie and advertising labels, and capped at 50.

The vision model is used only when the planner returns need_help, by choice or because its call failed. The agent

sends a screenshot and receives an action with a target point given as fractions of the 1280×800 viewport, where it clicks and, if needed, types. Such actions are not checked individually; a wrong click is caught, if at all, by the completion predicate. Identical screenshot-goal pairs are served from a cache.

C. Evidence-Gated Verification Model

We model an episode as a sequence of observations $o_t = \Omega(s_t)$, where s_t is the browser state after step t and Ω returns the page URL, the page title, the interactive-element manifest and a screenshot. The planner maps $(g, o_t, \mathcal{M})$, with g the user goal and $\mathcal{M}$ the retrieved context, to an action a_t . The executor applies a_t and returns a dispatch flag $\alpha_t \in \{0, 1\}$ that is 1 when the browser call returned without an exception or a failed low-level check (e.g., reading back the value of a text field).

a) Blocking predicate: $\beta(o) = 1$ when the observation shows a state in which further autonomous action is either impossible or not permitted: a `chrome-error://URL`, an HTTP error title (404, 500, 502, 503), or a CAPTCHA/bot-check page. A CAPTCHA, detected by a separate check that runs before Φ , forces the terminal state BLOCKED; the error cases make Φ false, so the episode continues.

b) Step predicates: The decomposer produces a plan $P = (g_1, \dots, g_n)$. In the current implementation the per-step predicate is weak: $\phi_i(o_{t+1}) := \alpha_t$, i.e., a step is marked complete when its action dispatched successfully, and the plan index advances.

c) Task predicate: Completion is decided by a separate predicate $\Phi(g, o, \hat{y}, P)$ evaluated only when the planner emits `complete`, when the plan index reaches n , or when a requested text entry has been dispatched. With $\hat{y}$ the extracted answer,

$$\Phi = \neg\beta(o) \wedge \neg H(g, o) \wedge T(g, o) \wedge X(g, \hat{y}) \wedge R(P, \hat{y}), \quad (1)$$

where H is true when the goal requires search or extraction but o is still a search-engine home page; T requires that any literal text requested in g appears in an input, textarea or contenteditable element of the live DOM; X requires, for extraction goals, a non-empty structured extraction or an answer longer than 40 characters that is not a restated navigation instruction; and R requires that no plan step remains pending unless an answer has already been produced. Each conjunct is optional and is only instantiated when g triggers it.

d) Completion rule: The agent reports success iff $\Phi = 1$, and every evaluation of Φ is written to the evidence store together with its expected and observed values. A rejected evaluation does not terminate the episode; control returns to observation and planning until a budget of $K = 15$ model calls (intent parsing, decomposition and planning) is exhausted. A final gate re-checks that navigation actually occurred when the intent named a target site, and downgrades the outcome to FAILED otherwise.

e) Property: Dispatch success does not imply task success:

$$\alpha_t = 1 \not\Rightarrow \Phi(g, o_{t+1}, \hat{y}, P) = 1. \quad (2)$$

A successful `navigate` to a search engine home page is the canonical counterexample: $\alpha_t = 1$ but $H = 1$, hence $\Phi = 0$.

f) False completion rate: Let $\mathcal{T}_c$ be the set of episodes the agent reports as completed, and let Φ^* be an independent, task-specific oracle predicate written by the experimenter (not by the agent). Then

$$\text{FCR} = \frac{|\{\tau \in \mathcal{T}_c : \Phi^*(\tau) = 0\}|}{|\mathcal{T}_c|}. \quad (3)$$

FCR is defined over reported completions, so it measures how often the agent claims success it has not achieved; task success rate is reported separately.

D. Failure Classification and Recovery

A failed action is first retried through a fresh observation by the `verify` node, up to three times. Later failures go to the recovery node, which matches the error text against patterns for seven types (`captcha`, `transient`, `element_not_found`, `verification_failed`, `navigation_failed`, `vision_needed`, `llm_failure`) or `unknown`. A CAPTCHA ends the task as BLOCKED. Other failures are assigned a strategy from the ladder `retry` $\rightarrow$ `alternative_selector` $\rightarrow$ `vision_fallback` $\rightarrow$ `replan`, one level per two attempts, with type-specific shortcuts; after six attempts the task fails. Each decision is stored as a recovery record. In the current code the chosen strategy is recorded but not executed: the strategy field is not declared in the state schema and is dropped, so every non-terminal recovery re-observes and re-plans.

E. Role-Based Local Model Routing

A deterministic analyzer maps each call's type, text and flags to required capabilities and a role: recovery calls to a recovery model; image inputs or screen-related wording to a vision model; intent parsing and simple navigation to a lightweight model; decomposition and multi-clause requests to a planner; the rest to a general model. Candidates per role come from a configured list (Table II) filtered by the models the Ollama server reports as installed (refreshed every 300 s). To limit model reloads, the router keeps the active model whenever it has the required capabilities, except for recovery calls; with no installed candidate it falls back to a fallback and then a base model. A single-model mode serves as an ablation. A switch cool-down is configurable but not read by the router.

F. Episodic Memory

On termination, `complete` writes the outcome to SQLite and to a Chroma collection: for a success, the goal, final URL and number of model calls; for a failure, the error and URL. The retrieval side exists: a context router queries memory (and, for longer or procedural tasks, a knowledge collection), and a builder deduplicates the results and fits them into a 3,500-token

TABLE II

DEFAULT ROLE CONFIGURATION OF THE MODEL ROUTER (OLLAMA TAGS, IN PREFERENCE ORDER). THE ROUTER USES THE FIRST CANDIDATE THAT IS INSTALLED. THE LAUNCHER INSTALLS ONLY `qwen2.5:1.5b` AND ONE VISION MODEL BY DEFAULT, IN WHICH CASE EVERY TEXT ROLE RESOLVES TO `qwen2.5:1.5b`.

Role	Selected when	Candidates
Lightweight	intent parsing; simple navigation	deepseek-r1:1.5b, qwen3.5:2b, qwen2.5:1.5b
Planner	decomposition; multi-clause or long requests	qwen3.5:2b, qwen2.5:7b, deepseek-r1:1.5b
Vision	image input or screen keywords	qwen3-vl:2b, moondream
Coder	coding keywords	qwen2.5-coder:3b, qwen2.5-coder:1.5b
Recovery	error recovery	qwen2.5:7b, qwen3.5:2b
General	otherwise	qwen2.5:1.5b, qwen2.5:7b
Fallback	no candidate installed	qwen2.5:1.5b

budget inside an untrusted-content banner in the planning prompt. On the live path, however, the runner initializes the retrieved lists as empty rather than unset, which the router treats as already retrieved; memory is written but not read. Since the stored records hold no selectors or action sequences, repairing the read path is necessary but not sufficient for learning from past runs.

G. Extension to Local System Access

The browser loop carries over to the desktop if the accessibility tree replaces the DOM, a window screenshot replaces the page screenshot, and typed operating-system predicates replace URL and DOM checks. Today the repository contains a desktop executor wrapping PyAutoGUI (click, type, hotkey, screenshot) and psutil (process listing); no node, action type or API route calls it, its only test checks that its methods exist, and without a display it silently returns a fixed screen size and empty screenshots. A file helper reads and writes text files but is not called by the agent and does not confine paths to its workspace. The rest of this subsection is a design (PROPOSED).

a) Actions and grounding: File actions (list, hash, read, create, overwrite with backup, copy, move, trash, delete), process actions (list, launch an allow-listed executable, terminate), application actions (open, focus, close) and UI actions (invoke, set value, type, hotkey). Grounding reads the foreground window’s accessibility tree through Windows UI Automation, the macOS Accessibility API or AT-SPI into a manifest with the same shape and budget as the DOM manifest, with element names sanitized. Vision on a window screenshot is used when the tree exposes too few actionable elements (canvas or GPU-drawn interfaces) or the planner asks for help, and a coordinate action counts as done only after its postcondition holds.

b) Verification: Every mutating action carries typed, exact postconditions evaluated after execution and stored with

TABLE III

PROPOSED DESKTOP POSTCONDITIONS AND DEFAULT PERMISSION TIERS (DESIGN, NOT IMPLEMENTED). h_0, t_0 : VALUES CAPTURED BEFORE THE ACTION; (pid, t_c) : PROCESS ID AND CREATION TIME.

Action	Postcondition checked after execution	Tier
file read, list, hash	workspace manifest unchanged	T0
create file	file exists $\wedge$ sha256 = sha256(content)	T1
overwrite	backup of pre-image stored $\wedge$ hash equals new content	T1
move	source absent $\wedge$ sha256(dst) = sha256(src)	T1
move to trash	path absent $\wedge$ trash entry with that origin	T1
permanent delete	path absent	T2
launch app	pid alive with creation time $t_c \wedge$ window title matches (pid, t_c) gone within timeout	T1/T3
terminate	foreground title matches pattern	T1/T2
focus window	accessibility value of element equals input	T0
set value	accessibility value of element equals input	T1/T3
toggle control	element state contains <i>checked</i>	T1/T2

before/after values (Table III). File predicates compare SHA-256 digests, sizes and modification times; process predicates identify a process by PID and creation time, which guards against PID reuse; window and accessibility predicates compare titles, focus, values and states. Read-only actions must leave a manifest of the workspace unchanged.

c) Permission tiers and enforcement: Tiers are computed from the concrete action and arguments, never from the model’s risk label. T0 (observe) runs freely. T1 (reversible: the inverse is recorded, e.g., a backed-up overwrite or a move to trash) runs automatically only if the user enabled automatic approval of low-risk actions. T2 (destructive: permanent deletion, terminating a process the agent did not start, invoking controls labelled Delete, Discard, Send or Pay) and T3 (network-capable programs, password fields) always need approval; secrets are injected by the executor and never pass through the model. Paths outside allow-listed roots, credential directories, inline-code interpreters and privilege elevation are refused. A new `action_gate` node between planning and execution applies the tier, a path jail that resolves symbolic links before the containment test, and a dry-run preview; approvals are bound to a hash of the exact action so arguments cannot change after consent. Each local action is appended to a hash-chained audit log that stores digests of arguments and state, never file contents or typed text.

V. IMPLEMENTATION

The backend is Python (10,755 lines, 1,682 of them in the graph nodes) on FastAPI, LangGraph, Playwright and Pydantic, with SQLite for tasks and events and a persistent local Chroma store for vectors. The client is React in a Tauri shell; the Observatory is a separate FastAPI service with a React dashboard. The code is public at <https://github.com/sudeshsudhii/Agentic-pilot-local>. **[TODO: AUTHOR: con-**

firm license; archive the evaluated version (tagged release with DOI).]

a) Design decisions: The runner records each node’s name and status in a trace and checkpoints the state after every node, so a task resumed after a pause, an approval or a CAPTCHA re-enters the graph with its saved plan and skips intent parsing. The completion decision lives in one node; the terminal node adds a guard that downgrades a completion to FAILED if the intent named a site but navigation was never verified. Planner output always passes through deterministic rules, which corrects small models on known patterns but places part of the behavior outside the model.

b) Evidence: Per task, the evidence directory holds before/after screenshots of each executed action, screenshots after navigation and on verified completion, the latest DOM manifest, append-only verification and recovery records, a per-action execution record (action, URL and screenshots before and after, result, model and role, duration) and the node trace. Step files are indexed by the model-call counter, so a retry at the same counter overwrites the earlier pair; files carry no integrity protection.

c) Status audit: Agent documentation tends to run ahead of the code, so we traced each mechanism to the path that executes it (Table IV). Most are live. Four are implemented but inert in a normal run: recovery strategies other than re-planning, memory retrieval, the routing cool-down and the verification ablation switch. The repository’s evaluation command calls scripts that are not in the repository, its ablation runner imports a function that does not exist, and its certification report counts a task as passed when it ends as completed and any screenshot and trace file exist. We use none of these repository-reported figures as results.

VI. EXPERIMENTAL SETUP

We report two kinds of evidence. Component studies isolate the completion gate and the model router; they need neither a language model nor network access and were run for this paper. End-to-end runs need local models and live sites; we specify their protocol here and leave their numbers to be filled in on the target hardware.

A. Component Studies

a) Environment: Linux container, 4-core Intel Xeon at 2.8GHz, 15 GiB RAM, no GPU; Python 3.11.15, Playwright 1.56.0 with headless Chromium 141.0.7390.37, Lang-Graph 1.2.12; repository commit 2361d50. Both study scripts are in the repository under `artifacts/experiments/`.

b) Gate study: We built 35 terminal browser states in six categories (Table V): search goals, navigation goals (including HTTP errors, a soft 404 and a login redirect), bot-check pages, exact text entry, extraction of a known fact, and two-step plans. Each state consists of a goal string, a URL, an HTML document served for that URL through Playwright request interception (no network traffic), and, where relevant, the extracted answer and the plan with step statuses. For each state we wrote an oracle label Φ^* stating whether the goal

TABLE IV
STATUS OF EACH MECHANISM AT COMMIT 2361d50, TRACED THROUGH THE CODE. I+T: IMPLEMENTED AND UNIT-TESTED; I: IMPLEMENTED, UNTESTED; P: PROPOSED OR PARTIAL. LIVE: AFFECTS THE DEFAULT EXECUTION PATH.

Mechanism	Status	Live	Note
Graph, checkpoints, resume	I+T	yes	compiled graph not tested
Completion predicate Φ	I+T	yes	rule-based (Sec. VII)
Per-step predicates, click check	P	–	steps advance on dispatch
CAPTCHA halt, resume, fallback	I+T	yes	no solver
DOM grounding, vision fallback	I+T	yes	vision on <code>need_help</code> only
Failure classes, retry bounds	I+T	yes	7 types, 6 attempts
Strategy-specific recovery	P	no	strategy label dropped
Role routing, stickiness	I+T	yes	cool-down not read
Memory write / retrieval	I / I+T	yes / no	retrieval bypassed
Sanitizer on element text	I+T	yes	title, URL unfiltered
Task-level approval gate	I	yes	model-assigned risk
Model-call audit log	I+T	yes	browsing not logged
Evidence store	I+T	yes	some files overwritten
Ablation flags	I	partial	verification flag unread
Desktop executor / extension	I / P	no	not wired (Sec. IV-G)

is satisfied, with a one-line rationale. Every case ends with a dispatched action, so a dispatch-only agent would report all of them complete. The evidence gate is the agent’s own code: the CAPTCHA pre-check of the `verify` node followed by `verify_task_completion`. Decisions are deterministic; latency is the median over 20 repetitions per case after one warm-up call. The cases and labels were written by the authors, are not sampled from real agent runs, and include deliberately difficult cases, so the resulting rates characterize the gate’s coverage rather than its expected performance.

c) Routing study: We marked a set of models as installed in the registry and called the router with the arguments used at each call site in the agent, both in isolation and in the agent’s call order with the previous selection passed as the active model.

B. End-to-End Protocol

a) Hardware and models: Record CPU, GPU and VRAM, RAM, operating system, Ollama version and the digest of every model used. The default configuration uses `qwen2.5:1.5b` for text and `moondream` for vision; a second configuration installs all models of Table II. The gateway currently sends no sampling options to Ollama, so each model’s default temperature applies; for reproducibility we set temperature to 0 and a fixed seed through the request options, a two-line change, and run three trials per task. **[TODO: MEASURE: hardware, OS, Ollama version and model digests.]**

b) Task suite: The benchmark list shipped in the repository has eight tasks, four of which lack a checkable success criterion. We therefore propose 30 tasks in six categories of five: navigation, search, text entry into a form, extraction of a fact

with a known answer, two- to three-step tasks, and blocking or error conditions (a public CAPTCHA demonstration page, HTTP error endpoints). Each task is paired with an oracle Φ^* written before any run, evaluated on the retained browser page or on the saved final screenshot and DOM snapshot by a person who does not see the agent’s reported status. **[TODO: AUTHOR: finalize the 30 tasks and their oracles; publish them with the code.]**

c) *Configurations*: (i) Full system. (ii) No completion gate: Φ replaced by “accept”, which requires making the `verify` node honor the existing but unused `enable_verification` flag. (iii) No recovery: `enable_recovery=false`, supported as is. (iv) Vision-only grounding: the DOM manifest withheld so that every action goes through the vision model, a small change. (v) Single model: supported as is. A memory ablation is meaningful only after the retrieval path is repaired (Section IV-F); until then, (vi) no memory is identical to (i).

d) *Metrics*: Task success rate under Φ^* ; FCR (Eq. 3); mean number of model calls and executed actions; mean wall-clock time per task; recovery success rate (tasks that entered recovery and ended satisfied under Φ^*); vision-invocation rate (fraction of actions grounded by the vision model); and evidence completeness (fraction of executed actions with both screenshots, an execution record and, for completed tasks, a completion-proof screenshot). All are computable from the task database and the evidence directory. Rates are reported with Wilson 95% intervals over task–trial pairs.

VII. RESULTS AND DISCUSSION

A. Completion Gate on Constructed End States

Table V reports the gate study. Of the 22 end states that do not satisfy their goal, the gate withholds completion in 15: all three search-engine home pages, all three pages with an HTTP error code in the title, three of the four bot-check pages (halted as BLOCKED), an empty and a wrong text field, three incomplete extraction answers including one produced while still on the search home page, and a plan with a pending step. The dispatch-only rule reports all 22 as complete. Among the 13 satisfied end states, the gate rejects one.

The FCR values in Table V depend on the mix of cases we chose and should not be read as expected rates on real tasks; the case-level outcomes are the informative part. The seven false completions each trace to a specific gap in Φ :

- *Wrong query (S7)*. A results page for a different query passes, because the search check only asks whether the URL is a results page.
- *Soft errors (N6, N7)*. A “page not found” message without a status code in the title, and a redirect to a login page, pass; neither is in the error vocabulary. The DOM extractor classifies login and error pages separately, but Φ does not consult that classification.
- *Unlisted challenge (C4)*. An interstitial whose text contains none of the 12 challenge phrases passes.
- *Substring match (T4)*. “Meeting at 10am tomorrow” satisfies a request to enter exactly “Meeting at 10am”.

TABLE V

COMPLETION DECISIONS ON 35 CONSTRUCTED END STATES. n^+/n^- : CASES WHOSE GOAL IS / IS NOT SATISFIED BY THE ORACLE. REP.: CASES REPORTED COMPLETE. FC: FALSE COMPLETIONS (REPORTED COMPLETE, GOAL NOT SATISFIED). FR: FALSE REJECTIONS (GOAL SATISFIED, NOT REPORTED COMPLETE). DISPATCH-ONLY REPORTS EVERY CASE COMPLETE. SOURCE: `VERIFICATION_GATE_SUMMARY.CSV`.

Category	n^+ n^-		Dispatch-only		Evidence gate		
			Rep.	FC	Rep.	FC	FR
Search	3	4	7	4	4	1	0
Navigation	2	5	7	5	4	2	0
CAPTCHA	0	4	4	4	1	1	0
Text entry	3	3	6	3	4	1	0
Extraction	3	4	7	4	3	1	1
Multi-step	2	2	4	2	3	1	0
All	13	22	35	22	19	7	1
FCR				0.629		0.368	

- *Length as a proxy for content (X4)*. Any answer longer than 40 characters that does not start with a navigation verb counts as an extraction, even if it lacks the requested fact. Because the planner’s own `reasoning` string on a `complete` action becomes the answer, a verbose planner can satisfy this check by itself.
- *Step status from dispatch (P3)*. A click that dispatched on the wrong element marks its step complete, so the plan check passes.

The single false rejection (X6) is the mirror image of X4: a correct answer of 13 characters is rejected by the length threshold. Each of these gaps corresponds to a check that can be added without a model call (query comparison, reuse of the page-state classifier, exact match for exact-text requests, answer-to-goal term overlap, and step predicates derived from the planner’s expected outcome), and the fixture set is designed to serve as a regression suite for them.

The gate is cheap relative to model inference. On a 4-core Intel Xeon (2.8GHz) container with Chromium 141 in headless mode, the median time for one full decision (CAPTCHA pre-check plus Φ) over 20 repetitions per case was 33.8ms, with a maximum per-case median of 52.5ms; decisions that stop at the URL-based CAPTCHA check took under 8ms. Most of the time is spent reading page content and title through the browser protocol.

B. Routing Behavior

Table VI shows which model the router selects for each call type the agent issues, evaluated without inference for two sets of installed models. With the launcher’s default installation, every text call resolves to `qwen2.5:1.5b`, three of them through the fallback chain, so role-based routing reduces to a single text model and a vision model. With all configured models installed and each call routed in isolation, each role receives its preferred model. When the calls are issued in the order the agent issues them, passing the previously selected model as the active one, stickiness keeps the lightweight model chosen for intent parsing (`deepseek-r1:1.5b`) for task decomposition and multi-step planning, because the router’s

TABLE VI

MODEL SELECTED BY THE ROUTER PER CALL TYPE (NO INFERENCE PERFORMED). DEFAULT: QWEN2.5:1.5B AND MOONDRREAM INSTALLED. FULL: ALL CONFIGURED MODELS INSTALLED. ISOLATED: EACH CALL ROUTED WITH NO ACTIVE MODEL. AGENT ORDER: CALLS ROUTED IN GRAPH ORDER WITH THE PREVIOUS SELECTION AS THE ACTIVE MODEL. ^fSELECTED BY THE FALLBACK CHAIN. SOURCE: ROUTING_RESOLUTION.CSV.

Call (role)	Default	Full, isolated	Full, agent order
Intent parsing (light.)	qwen2.5:1.5b	deepseek-r1:1.5b	deepseek-r1:1.5b
Decompose (planner)	qwen2.5:1.5b ^f	qwen3.5:2b	deepseek-r1:1.5b
1-step act. (light.)	qwen2.5:1.5b	deepseek-r1:1.5b	deepseek-r1:1.5b
Multi-step act. (planner)	qwen2.5:1.5b ^f	qwen3.5:2b	deepseek-r1:1.5b
Recovery (recovery)	qwen2.5:1.5b ^f	qwen2.5:7b	qwen2.5:7b
Image input (vision)	moondream	qwen3-vl:2b	qwen3-vl:2b

capability profile lists it as able to plan. Only recovery, which is exempt from stickiness, and image inputs switch models. Stickiness thus saves model loads at the cost of never using the planner model in a normal run. Whether that trade is worth making is an empirical question for the ablations of Section VI.

C. Test Suite

[TODO: MEASURE: test-suite table (pending evaluation run).]

D. End-to-End Evaluation

End-to-end results require a local Ollama installation and live websites, neither of which was available in the environment used to prepare this draft. **[TODO: MEASURE: run the protocol of Section VI and report, per configuration, task success rate, FCR under the independent oracle, mean model calls, mean wall-clock time per task, recovery success rate, vision-invocation rate and evidence completeness, with 95% confidence intervals over trials.] [TODO: MEASURE: add a qualitative case study with before/after screenshots from the evidence directory (~/.pilot/logs/evidence/<task_id>/) of one task in which the gate rejected a premature completion and the agent then finished the task.]**

VIII. SECURITY, PRIVACY AND ETHICAL CONSIDERATIONS

a) Threat model: We consider (i) instructions embedded in page content that try to redirect the agent [27]; (ii) high-impact actions such as posting, sending or purchasing without consent; (iii) damage to the local system once file and process actions exist; and (iv) disclosure of user data to unintended parties. The user and the host are trusted; websites are not. A compromised host or model is out of scope.

b) Current mitigations: A sanitizer replaces 17 patterns of instruction-override, delimiter, exfiltration and role-manipulation phrasing in element text, labels and placeholders before they reach the planner; retrieved knowledge is filtered the same way and marked untrusted. Before navigation, a risk gate pauses tasks whose action is posting, e-mailing,

purchasing, deleting or transferring, or whose risk level is high or critical, and asks the user; the check is repeated when completion is verified. The service listens on loopback and accepts only localhost origins. Every model call is logged with destination, a locality flag and payload sizes, and credential-like values are scrubbed from broadcast events.

c) Gaps: The sanitizer is a denylist and can be evaded by paraphrase or encoding [28], and the page title, URL, extracted text and screenshots are not filtered. The risk level that triggers approval comes from the same model that reads untrusted content, and one approval covers every later action of the task. The audit log covers model calls but not browsing, and a non-local model address is logged, not blocked. The privacy property we claim is therefore narrow: *by default, model inference runs on the user’s machine and no prompt or screenshot is sent to a model provider.* Data the user asks the agent to enter on a website still leaves the machine. The desktop design closes some of these gaps with deterministic tiers, action-bound approvals and a hash-chained audit log.

d) CAPTCHAs and automation etiquette: On a CAPTCHA or bot check the agent stops, marks the task BLOCKED, keeps the browser open, and lets the user either solve the challenge and resume, which is refused while the challenge is still present, or restart on another search engine. There is no solver. The browser is, however, launched with automation-detection features disabled, a desktop user-agent string and `navigator.webdriver` hidden, and direct loads of search-result URLs are replaced by typed queries. This is defensible for a personal assistant acting at human pace on its user’s requests, but not for bulk use, and deployments should follow site terms of service. Chromium also runs with its sandbox disabled, which weakens isolation from hostile pages; re-enabling it is a simple hardening step.

IX. LIMITATIONS AND FUTURE WORK

a) Evidence: Only the completion gate and the router are measured here, and the gate only on constructed end states. End-to-end rates and latencies with local models remain to be measured (Section VI); latency will depend strongly on the host and on which models are installed.

b) Rule-based verification: Φ is keyed to request wording and a few search engines, and Section VII lists its blind spots. We plan to derive typed step predicates from the planner’s expected outcomes, add a local vision-model check of the final screenshot as a second opinion, and calibrate both on labelled trajectories.

c) Small models and hand-written rules: Planning with 1.5B–7B models is weak, and deterministic overrides compensate on common patterns, some of them site-specific. The extraction handler also contains post-processing written for one case-study site. **[TODO: AUTHOR: remove or generalize the case-study-specific extraction code in `backend/agent/nodes.py` (lines 1108–1139) before any end-to-end evaluation, or disclose it here.]** Canvas- and WebGL-rendered pages expose no useful DOM and depend on the vision path.

d) *Inactive mechanisms and roadmap*: Strategy-specific recovery, memory retrieval, the routing cool-down and the verification switch need small fixes to take effect (Table IV); the ablations are meant to measure them once they do. For the local-system extension we will implement the path and process policy, the predicate library, an AT-SPI provider (testable headlessly on Linux) and the `action_gate` node, then Windows UI Automation. Longer term, we plan to split planning, grounding and verification across cooperating agents, with the verifier given no write access to the environment.

X. CONCLUSION

Agentic Pilot runs a browser agent entirely against local models and refuses to report success until a check on the observed page passes, keeping the observations behind each decision on disk. Its architecture separates observation, planning, execution, verification and recovery into explicit graph nodes, which made it possible to state the completion rule formally and to test it in isolation. On constructed end states the rule removes most of the premature completions a dispatch-only agent would report, and its failures point to specific checks that are missing. The same structure carries over to files, processes and native applications once accessibility trees, typed postconditions and deterministic permission tiers take the place of the DOM, page checks and model-assigned risk. The remaining work is empirical: measuring the full system with local models on real tasks, and activating the recovery and memory paths that are designed but not yet live.

ACKNOWLEDGMENT

[TODO: AUTHOR: review and adapt.] An AI system (Anthropic’s Claude) assisted with analyzing the source code, running the reported component experiments, searching and checking references, and drafting the text of this paper. The authors reviewed, verified and revised all content and take full responsibility for it.

REFERENCES

- [1] H. He, W. Yao, K. Ma, W. Yu, Y. Dai, H. Zhang, Z. Lan, and D. Yu, “WebVoyager: Building an end-to-end web agent with large multimodal models,” in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL) (Volume 1: Long Papers)*, 2024, pp. 6864–6890.
- [2] B. Zheng, B. Gou, J. Kil, H. Sun, and Y. Su, “GPT-4V(ision) is a generalist web agent, if grounded,” in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 235, 2024, pp. 61 349–61 385.
- [3] C. Zhang, L. Li, S. He, X. Zhang, B. Qiao, S. Qin, M. Ma, Y. Kang, Q. Lin, S. Rajmohan, D. Zhang, and Q. Zhang, “UFO: A UI-focused agent for Windows OS interaction,” in *Proc. 2025 Conf. Nations of the Americas Chapter Assoc. Comput. Linguistics: Human Language Technologies (NAACL-HLT) (Volume 1: Long Papers)*, 2025, pp. 597–622.
- [4] C. Zhang, Z. Yang, J. Liu, Y. Li, Y. Han, X. Chen, Z. Huang, B. Fu, and G. Yu, “AppAgent: Multimodal agents as smartphone users,” in *Proc. 2025 CHI Conf. Human Factors Comput. Syst. (CHI)*, 2025, pp. 70:1–70:20.
- [5] N. Lukas, A. Salem, R. Sim, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, “Analyzing leakage of personally identifiable information in language models,” in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2023, pp. 346–363.
- [6] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, “WebArena: A realistic web environment for building autonomous agents,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.
- [7] T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, T. J. Hua, Z. Cheng, D. Shin, F. Lei, Y. Liu, Y. Xu *et al.*, “OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, vol. 37, 2024.
- [8] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, “Large language models cannot self-correct reasoning yet,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.
- [9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.
- [10] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 24 824–24 837.
- [11] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023.
- [12] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023.
- [13] G. Kim, P. Baldi, and S. McAleer, “Language models can solve computer tasks,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023.
- [14] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, “Voyager: An open-ended embodied agent with large language models,” *Trans. Mach. Learn. Res. (TMLR)*, 2024.
- [15] T. Shi, A. Karpathy, L. Fan, J. Hernandez, and P. Liang, “World of bits: An open-domain platform for web-based agents,” in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 70, 2017, pp. 3135–3144.
- [16] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su, “Mind2Web: Towards a generalist agent for the web,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, vol. 36, 2023.
- [17] Z. Liu, C. Chen, J. Wang, M. Chen, B. Wu, X. Che, D. Wang, and Q. Wang, “Make LLM a testing expert: Bringing human-like interaction to mobile GUI testing via functionality-aware decisions,” in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE)*, 2024, pp. 100:1–100:13.
- [18] Z. Liu, C. Chen, J. Wang, X. Che, Y. Huang, J. Hu, and Q. Wang, “Fill in the blank: Context-aware automated text input generation for mobile GUI testing,” in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, 2023, pp. 1355–1367.
- [19] J. Yang, H. Zhang, F. Li, X. Zou, C. Li, and J. Gao, “Set-of-mark prompting unleashes extraordinary visual grounding in GPT-4V,” arXiv:2310.11441, 2023.
- [20] W. Hong, W. Wang, Q. Lv, J. Xu, W. Yu, J. Ji, Y. Wang, Z. Wang, Y. Dong, M. Ding, J. Tang *et al.*, “CogAgent: A visual language model for GUI agents,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 14 281–14 290.
- [21] K. Cheng, Q. Sun, Y. Chu, F. Xu, L. YanTao, J. Zhang, and Z. Wu, “SeeClick: Harnessing GUI grounding for advanced visual GUI agents,” in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL) (Volume 1: Long Papers)*, 2024, pp. 9313–9332.
- [22] Y. Lu, J. Yang, Y. Shen, and A. Awadallah, “OmniParser for pure vision based GUI agent,” arXiv:2408.00203, 2024.
- [23] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [24] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. 2019 Conf. Empirical Methods Natural Lang. Process. and 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.
- [25] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” in *Proc. 36th Annu. ACM Symp. User Interface Softw. Technol. (UIST)*, 2023.
- [26] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, “MemGPT: Towards LLMs as operating systems,” arXiv:2310.08560, 2023.
- [27] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-

- integrated applications with indirect prompt injection,” in *Proc. 16th ACM Workshop Artif. Intell. Secur. (AISec)*, 2023.
- [28] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Formalizing and benchmarking prompt injection attacks and defenses,” in *Proc. 33rd USENIX Secur. Symp. (USENIX Security)*, 2024, pp. 1831–1847.
 - [29] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, “InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents,” in *Findings Assoc. Comput. Linguistics: ACL 2024*, 2024, pp. 10 471–10 506.
 - [30] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, vol. 37, 2024.
 - [31] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu *et al.*, “Qwen2.5 technical report,” arXiv:2412.15115, 2024.
 - [32] P. Wang, S. Bai, S. Tan, S. Wang, Z. Fan, J. Bai *et al.*, “Qwen2-VL: Enhancing vision-language model’s perception of the world at any resolution,” arXiv:2409.12191, 2024.
 - [33] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, “AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration,” in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 6, 2024.
 - [34] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, “A survey on large language model based autonomous agents,” *Front. Comput. Sci.*, vol. 18, no. 6, p. 186345, 2024.
 - [35] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong *et al.*, “The rise and potential of large language model based agents: A survey,” *Sci. China Inf. Sci.*, vol. 68, p. 121101, 2025.